

RANK WHERE IT COUNTS: EXTENDING LoRA ON GPT-2

1 INTRODUCTION

We re-implement LoRA from scratch¹ on GPT-2 Medium, then ask two follow-up questions the original paper does not address. Full fine-tuning of GPT-2 Medium (354M parameters) updates every weight and stores a fresh 354M checkpoint per task, which is expensive to train, store, and serve when many downstream tasks are needed. LoRA (Hu et al., 2021) addresses this. It freezes the pre-trained weights W_0 and trains a small rank- r matrix product $B \cdot A$ (rank r much smaller than hidden dimension d) in parallel with each frozen weight. At inference, $\Delta W = B \cdot A$ folds back into W_0 , so the deployed model has the same shape and the same latency as the original. One of the paper’s central claims is that on GPT-2 Medium, LoRA matches full fine-tuning at roughly 0.1% of the trainable parameters and zero added latency.

We ask two follow-up questions. **Q1: Can LoRA itself be made better under the same rank budget?** LoRA gives every transformer layer the same rank. We hypothesize that later layers need more rank than early layers, because early layers learn general syntax and structure, while later layers learn the more intricate, task-specific patterns. We test this two ways: a hand-designed late-heavy schedule (Late-Ramp), and a learned-and-pruned schedule (AdaLoRA-Lite). **Q2: Can one LoRA-based model be good at every task?** A single LoRA is strong on its training task but collapses off-domain. We train a Mixture-of-LoRA-Experts router (Wu et al., 2024) (one router that picks among many task-specific LoRAs per token) over three frozen single-task LoRAs.

2 CHOSEN RESULT

We aim to reproduce the GPT-2 Medium row of LoRA’s Table 3 (Hu et al., 2021): GPT-2 Medium with LoRA at rank $r=4$ on the query (Q) and value (V) attention projections, fine-tuned on the E2E NLG Challenge (Novikova et al., 2017) (a data-to-text dataset). The paper reports **0.35M trainable parameters** (0.1% of GPT-2 Medium) and **BLEU 70.4 $\pm$ 0.1** (NIST 8.85, METEOR 46.8, ROUGE-L 71.8, CIDEr 2.53). Table 1 (Appendix A) reproduces Table 3 verbatim. We chose this number because it is LoRA’s central claim: match or beat full fine-tuning at 0.1% of the trainable parameters.

3 METHODOLOGY

Reproduction. We implement LoRA from scratch in PyTorch (no `peft`, no `loralib`). The backbone is `gpt2-medium` (24 layers, 354M params), with all weights frozen. Each layer’s fused `c_attn` convolution (GPT-2’s single tensor producing Q, K, V together) is sliced so LoRA attaches only to Q and V (matching paper). The adapter is initialised with $A \sim \mathcal{N}(0, \sigma^2)$ (Kaiming-uniform) and $B = 0$, so $\Delta W = 0$ at start. We use scaling α/r with $\alpha = 32$ and dropout 0.1. Training follows the paper: AdamW, learning rate $2e-4$, linear decay, 500-step warmup, batch 8, 5 epochs, label smoothing 0.1. The one modification is fp16 mixed precision (paper used fp32), forced by Colab memory. Decoding is beam search (beam=10, length penalty 0.9, no-repeat-4-gram); crucially the EOS list includes both 50256 (end-of-text) and 628 (E2E newline) to match the official LoRA repository. Evaluation uses the official `e2e-metrics` scripts.

Q1 extensions (same rank budget as uniform $r=4$: $4 \times 2 \times 24L = 192$ -rank-units). Both methods test the same hypothesis: rank should be spent late, not uniformly. **(a) Late-Ramp** (hand-designed) sets $r \in \{2, 3, 4, 4, 5, 6\}$ from early blocks to late blocks; parameter count is identical to uniform $r=4$. **(b) AdaLoRA-Lite** (learned) is a lightweight variant of AdaLoRA (Zhang et al., 2023). We attach a scalar gate s_i to each rank direction and update an EMA (exponential moving average) of its sensitivity score $|s_i \cdot \partial L / \partial s_i|$ with $\beta = 0.85$. Every 500 steps we rank gates by this score and prune the least important along a cubic schedule, starting from over-rank $r_{\max} \in \{6, 8, 16\}$ and landing at $r = 4$, hence back to 192 active rank units before evaluation.

Q2 extension — MoLE. We train three single-task LoRAs (E2E, DART (Nan et al., 2021), WebNLG (Gardent et al., 2017)) at $r=4$, freeze them, then train only a per-layer top-1 router `Linear( $d \rightarrow 3$ )` (0.07M trainable params). At each token the router picks one expert. Training uses the union of the three task training sets; evaluation uses each task’s held-out test split.

¹<https://github.com/justinlxiang/CS4782-final-project>

4 RESULTS AND ANALYSIS

Reproduction. Our uniform $r=4$ implementation reaches BLEU 67.30 on E2E (Table 2). The deployed model has the same shape and the same per-latency as GPT-2 Medium, exactly as claimed by the original paper. We don’t match the paper’s reported BLEU 70.4, which we discuss in Section 5.

Q1(a) Hand-designed Late-Ramp. Reallocating the same 192 rank units toward later layers raises E2E BLEU from 67.30 to **68.39** at zero extra parameter cost (Table 2). Late-Ramp also beats uniform $r=16$ (1.57M, BLEU 68.21) on every official metric, which means it wins with $4\times$ fewer parameters. The hypothesis is supported: rank spent late beats rank spent uniform. **Q1(b) Learned AdaLoRA-Lite.** When we do not hand-design the schedule and let the gates learn which rank directions matter, the best run (AdaLoRA-Lite $r_{\max}=6$) reaches **BLEU 68.85** at the same 0.39M active-parameter budget (Table 2). This is above paper full fine-tuning (68.20) and roughly tied with the paper’s 11M-parameter Adapter^L baseline (68.9), so we match its quality at $28\times$ fewer trainable parameters. AdaLoRA-Lite $r_{\max}=6$ also outperforms uniform $r=16$ (1.57M, BLEU 68.21) on every metric. Importantly, the learned per-layer active-rank profile drops to 3–4 in the earliest blocks and stays at the cap of 10–12 in the later blocks; the profile is late-heavy—the model converged to our design.

Q1 takeaway. The hand-designed hypothesis (“rank should be spent late”) and the statistically-learned answer (gates prune away early-layers) settle on essentially the same strategy through completely different mechanisms. This means the late bias is not a hand-tuned trick; it is the structure the model finds on its own. We see this as strong evidence for the depth-specialization hypothesis: capacity matters more late than early, because later is where task-specific knowledge concentrates.

Q2: cross-task generalisation. Single-task LoRAs collapse off-domain (Table 3): the E2E specialist scores 67.30 BLEU on its own task but drops to 28.53 on DART and 10.58 on WebNLG. A single MoLE checkpoint (three already-trained frozen experts at ~ 1.17 M plus a 0.07M trainable router) recovers most of each specialist’s BLEU on its own data while remaining usable on all three: E2E 67.76, DART 45.35, WebNLG 51.75. This means one served model can in fact be made good at every task, with only an extra router. To understand the router’s behavior, we logged what fraction of each task’s tokens get routed to each expert at the end of training (Table 4). E2E and DART tokens prefer their own experts (49% and 74% argmax fraction respectively), but WebNLG tokens lean more on the DART expert (64%) than their own WebNLG expert (31%), which mechanistically explains why MoLE’s WebNLG BLEU (51.75) trails the WebNLG specialist’s 55.23 in Table 3.

5 REFLECTIONS

Our uniform $r=4$ run reaches BLEU 67.30, which is below the paper’s 70.4. We see two main plausible sources. First, we use fp16 rather than the paper’s fp32 (constraint by Colab memory), which is a noticeable performance drag. Second, we report a single seed while the paper averages over three; this means some seed noise plausibly affects performance. However, the direction of LoRA’s central claim reproduces: we match the paper’s reported full-finetuning performance with only fp16, single-seed LoRA.

Implementation lessons. Small details produced large metric changes; matching the official LoRA repository’s beam-search EOS list (both 50256 (end-of-text) and 628 (the E2E newline) as stopping tokens) alone moved BLEU by ~ 1 point, so reading the reference repository was essential. CPU unit tests caught adapter-wiring and masking bugs that would have been very slow to surface from a failed Colab training run. Budget allocation is a useful diagnostic, not just a metric trick: the convergence between Late-Ramp and AdaLoRA-Lite gave us a more confident statement about *where* LoRA’s capacity does work than any single BLEU number would.

What we would do differently, and future work. With more compute we would run each headline configuration at three seeds with mean $\pm$ standard deviation, because this would separate fp16 effects from seed noise. For future directions, we would scale the late-heavy hypothesis beyond GPT-2 Medium (Llama-2/3 7B, Mistral 7B), because the hypothesis is most interesting when early and late layers truly do different work. We would also attempt to compose Q1 and Q2 into MoLE + adaptive rank (top- k routing with per-expert AdaLoRA-Lite budgets), and apply LoRA / AdaLoRA-Lite to the FFN projections (W_{fc} , W_{proj}), because most of the parameter budget lives there.

REFERENCES

- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv:2106.09685*, 2021.
- Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. In *International Conference on Learning Representations (ICLR)*, 2023.
- Xun Wu, Shaohan Huang, and Furu Wei. Mixture of LoRA Experts. In *International Conference on Learning Representations (ICLR)*, 2024.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. In *International Conference on Machine Learning (ICML)*, 2019.
- Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.
- Jekaterina Novikova, Ondřej Dušek, and Verena Rieser. The E2E Dataset: New Challenges for End-to-End Generation. In *SIGDIAL*, 2017.
- Linyong Nan, Dragomir Radev, Rui Zhang, Amrit Rau, Abhinand Sivaprasad, Chiachun Hsieh, Xiangru Tang, Aadit Vyas, Neha Verma, Pranav Krishna, Yangxiaokang Liu, Nadia Irwanto, Jessica Pan, Faiaz Rahman, Ahmad Zaidi, Mutethia Mutuma, Yasin Tarabar, Ankit Gupta, Tao Yu, Yi Chern Tan, Xi Victoria Lin, Caiming Xiong, Richard Socher, and Nazneen Fatema Rajani. DART: Open-Domain Structured Data Record to Text Generation. In *NAACL*, 2021.
- Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. The WebNLG Challenge: Generating Text from RDF Data. In *International Natural Language Generation Conference (INLG)*, 2017.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-Art Natural Language Processing. In *EMNLP: System Demonstrations*, 2020.
- Ondřej Dušek. e2e-metrics: Official E2E NLG evaluation scripts. <https://github.com/tuetschek/e2e-metrics>, 2017.

A TABLES

This appendix contains the four reference tables cited in the body.

Table 1: Verbatim from Hu et al. (2021), Table 3. GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge. For all metrics, higher is better. LoRA outperforms several baselines with comparable or fewer trainable parameters. Confidence intervals are shown for experiments the original authors ran. * indicates numbers published in prior works. Our reproduction target is the bolded **GPT-2 M (LoRA)** row.

Model & Method	# Trainable Params	BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4 $\pm$.1	8.85 $\pm$.02	46.8 $\pm$.2	71.8 $\pm$.1	2.53 $\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49 $\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4 $\pm$.1	8.89 $\pm$.02	46.8 $\pm$.2	72.0 $\pm$.2	2.47 $\pm$.02

Table 2: Full results on E2E NLG test, official e2e-metrics. Top three rows are paper references for comparison (rows from Table 1). Middle five rows are our uniform-rank sweep. Bottom six rows are our two Q1 budget-allocation methods (Late-Ramp and AdaLoRA-Lite), each at three settings. AdaLoRA-Lite rows are reported at *active* rank budget; the allocated tensor count is larger but pruned down before evaluation. Bold marks our best run.

Configuration	Params	BLEU	NIST	METEOR	ROUGE-L	CIDEr
LoRA paper GPT-2 M $r=4$ (Hu et al., 2021)	0.35M	70.40	8.85	46.80	71.80	2.53
Full fine-tuning, paper	354.92M	68.20	8.62	46.20	71.00	2.47
Adapter ^L baseline, paper	11.09M	68.90	8.71	46.10	71.30	2.47
Uniform $r=1$ (ours)	0.10M	66.78	8.51	45.02	68.94	2.36
Uniform $r=2$ (ours)	0.20M	67.11	8.52	45.78	70.30	2.45
Uniform $r=4$ (ours, paper reproduction)	0.39M	67.30	8.51	46.05	70.96	2.47
Uniform $r=8$ (ours)	0.79M	67.97	8.59	46.23	70.94	2.50
Uniform $r=16$ (ours)	1.57M	68.21	8.62	46.22	71.12	2.49
Late-Ramp $r=4$ (Q1a, ours)	0.39M	68.39	8.63	46.51	71.29	2.49
Late-Ramp $r=8$ (ours)	0.79M	67.77	8.57	46.28	70.90	2.48
Late-Ramp $r=16$ (ours)	1.57M	68.15	8.62	46.38	71.24	2.49
AdaLoRA-Lite $r_{\max}=16$ (ours)	0.39M*	67.67	8.59	46.10	70.50	2.45
AdaLoRA-Lite $r_{\max}=8$ (ours)	0.39M*	68.34	8.64	46.30	71.00	2.49
AdaLoRA-Lite $r_{\max}=6$ (Q1b, ours, best)	0.39M*	68.85	8.69	46.40	71.42	2.51

Table 3: Cross-task BLEU (Q2). Each row is one trained adapter; each column is the held-out test split of that task. Single-task specialists collapse off-domain (off-diagonal cells), but MoLE stays strong across all three tasks from a single served checkpoint. Bold marks the best in each column.

Method	Adapter Params	E2E BLEU	DART BLEU	WebNLG BLEU
E2E LoRA specialist	0.39M	67.30	28.53	10.58
DART LoRA specialist	0.39M	60.54	47.15	50.44
WebNLG LoRA specialist	0.39M	39.75	33.41	55.23
MoLE (3 experts + router)	$\approx$ 1.24M	67.76	45.35	51.75

Table 4: MoLE router argmax distribution at the end of training (epoch 2), aggregated across all 24 GPT-2-Medium layers. Each row is a task’s token stream; each cell is the fraction of those tokens whose top-1 expert is the column expert (rows sum to 1). Bold marks each row’s preferred expert. E2E and DART tokens prefer their own experts, but WebNLG tokens lean on the DART expert over their own.

Task tokens	argmax $\rightarrow$ E2E expert	argmax $\rightarrow$ DART expert	argmax $\rightarrow$ WebNLG expert
E2E	0.49	0.35	0.16
DART	0.04	0.74	0.22
WebNLG	0.05	0.64	0.31